

PSE-TRAVERSE-DYNAMICS-01

Formale Spezifikation einer domänenagnostischen morphodynamischen
Traversierungsschicht

Extraktion des neutralen Kerns aus DioniceOS zur Implementierung in `pse-traverse`

Zweck. Dieses Dokument definiert einen implementierungsfähigen Blueprint für eine optionale dynamische Erweiterung von `pse-traverse`. Der Kern extrahiert aus DioniceOS ausschließlich die domänenneutralen Mechanismen: Zustandslift, Feldabsorption, Gradientenführung, morphodynamische Kompression, Policy-Regelung, Transition-Proof, Gate-Entscheidung und replayfähige Tick-Berichte. Alle symbolischen, metaphysischen, projektspezifischen und brandingbezogenen Begriffe werden aus dem Kern entfernt.

Version: 0.1.0

Status: Implementierungs-Blueprint / Agenten-Spezifikation

Sprache: Deutsch

Primärziel: deterministische Rust-Implementierung

Kompatibilität: `pse-traverse-core`, `pse-traverse-signature`, `pse-core`

Inhaltsverzeichnis

1	Executive Summary	3
2	Quellenbasis und Extraktionsprinzip	3
2.1	Relevante DioniceOS-Komponenten	3
2.2	Neutralisierungsregeln	4
3	Systemrolle innerhalb der Gesamtarchitektur	4
3.1	Layer-Modell	5
3.2	Integration in bestehende Pipeline	5
4	Normative Sprache	6
5	Kernabstraktionen	6
5.1	CanonicalNumber	6
5.2	Hash256 und StableId	6
5.3	BaseState und LiftedState	7
5.4	StateLift	7
6	Field Layer	8
6.1	FieldSignal	8
6.2	FieldAbsorber	8
7	Guidance Field	9
7.1	Datenmodell	9
7.2	GuidanceField-Trait	9
7.3	Relaxationsregeln	9
8	Morphodynamic Compressor	10
8.1	Kompressionsmodell	10
8.2	Kompressor-Trait	10
8.3	CompressorStats	11
8.4	Hebbian Update	11
8.5	Split	11
8.6	Merge	11
8.7	Prune	12
9	Dynamic Policy	12
9.1	Policy-Klassen	12
9.2	Semantik	12
9.3	Homeostasis-Adaption	12
10	Transition Proof und Dynamic Gate	13
10.1	TransitionProof	13
10.2	Berechnung	13
10.3	DynamicGate	13
11	Dynamic Tick	14
11.1	State Container	14

11.2 Input und Report	14
11.3 Algorithmus	15
12 Interaction mit Signature Layer	15
13 Integration mit pse-traverse-core	16
13.1 Erweiterte Run-Struktur	16
13.2 CollapsePlan-Einfluss	17
14 Replay und Determinismus	17
14.1 DynamicRunDescriptor	17
14.2 Stop Conditions	18
14.3 Replay-Regel	18
14.4 Nichtdeterminismus-Ausschlüsse	18
15 Fehlersemantik	18
16 CLI-Spezifikation	19
16.1 Subcommands	19
16.2 CLI-Anforderungen	19
17 Testplan	19
17.1 Unit Tests	19
17.2 Integration Tests	19
17.3 Property Tests	20
18 Definition of Done	20
19 Empfohlene Dateistruktur	21
20 Coding-Agent-Auftrag	21
21 Abschließende Architekturformel	22
A Glossar	23
B Konformitätsmatrix	23

1 Executive Summary

Diese Spezifikation definiert `pse-traverse-dynamics` als optionale, deterministische Dynamikschicht für `pse-traverse-core`. Sie überführt statische Traversierungszustände in einen getakteten, feldgekoppelten und komprimierenden Prozess. Die Schicht ist nicht als neues Betriebssystem, nicht als neues Ledger, nicht als Ersatz für PSE und nicht als metaphysische Ontologie zu verstehen. Sie ist ein technischer Mechanismus:

**Zustand → Lift → Feldabsorption → Gradientenführung → Kompression
→ Transition-Proof → Gate → replayfähiger Bericht**

Der Beitrag von `pse-traverse-dynamics` besteht darin, dass `pse-traverse-core` nicht nur einen einmaligen `CollapsePlan` berechnet, sondern den zugrundeliegenden Problemraum überlogisch getaktete Zustandsübergänge stabilisieren kann. Dadurch entstehen neue Fähigkeiten:

- dynamische Verdichtung von Kandidaten- und Planräumen,
- Erkennung instabiler Übergänge durch Transition-Proofs,
- kontrollierter Wechsel zwischen Exploration, Exploitation und Homeostasis,
- Dichtekontrolle gegen Suchraumexplosion,
- Integration dynamischer Gate-Signale in bestehende `GateReports`,
- vollständig replayfähige Tick-Sequenzen ohne Wall-Clock-Abhängigkeit.

Strikte Abgrenzung

`pse-traverse-dynamics` darf keine `SemanticCrystals` erzeugen. Echte post-symbolische Crystals bleiben ausschließlich Aufgabe der vorhandenen PSE-Bridge in `pse-core`. `pse-traverse-dynamics` erzeugt nur Zustände, Transition-Proofs, Gate-Signale, Kompressionsstatistiken und Tick-Berichte.

2 Quellenbasis und Extraktionsprinzip

2.1 Relevante DioniceOS-Komponenten

Die technische Extraktion basiert auf folgenden in DioniceOS vorhandenen Strukturen:

DioniceOS-Komponente	Beobachteter Mechanismus	Neutraler Export
<code>trichter/types.rs</code>	<code>State4D</code> , <code>GuidanceVector</code> , <code>FunnelNode</code> , <code>ProofHash</code>	<code>BaseState</code> , <code>LiftedState</code> , <code>GuidanceState</code> , <code>CompressionNode</code> , <code>TransitionProofId</code>
<code>trichter/lift.rs</code>	Lift $\mathbb{R}^4 \rightarrow \mathbb{R}^5$ und Projektion zurück in den Prozessraum	<code>StateLift</code> : generisches $N \rightarrow N + 1$ -Lift/Projection-Protokoll

trichter/hyperbion.rs	Berechnung von Resonanzfeld und morphodynamischem Varianzfeld	FieldAbsorber: Alignment, Dispersion, Pressure
trichter/hdag.rs	gerichtetes Feld aus Tensoren, Transitionen, Relaxation und Gradientenberechnung	GuidanceField: Knoten, gerichtete Kohärenztransitionen, Gradient
trichter/funnel.rs	Kompressionsgraph mit Hebbian Update, Split, Merge, Prune	MorphodynamicCompressor
trichter/policies.rs	Explore, Exploit, Homeostasis mit Dichteadaption	DynamicPolicy
trichter/tick.rs	deterministischer Kopplungstick	dynamic_tick
adapters/state_adapter.rs	verlustarme Repräsentationskonversion mit Roundtrip-Test	RepresentationAdapter
adapters/resonance_adapter.rs	Proof-of-Resonance und FIRE/HOLD-Gate	TransitionProof und DynamicGate
unified/cognitive_engine.rs	8-Phasen-Pipeline: Integration, Signatur, Konversion, Routing, Knowledge, Proof, Gate, Output	DynamicTraversalPipeline ohne domänenspezifische Knowledge-Schicht

2.2 Neutralisierungsregeln

NEUTRAL-CORE-01

Der Kern dieser Spezifikation kennt keine DioniceOS-spezifischen Eigennamen, keine sakralen Geometrien, keine fest verdrahtete 4D/5D-Ontologie und keine externe Knowledge-Ledger-Semantik. Er kennt ausschließlich Zustände, Lifts, Felder, Gradienten, Kompression, Policies, Proofs, Gates und replayfähige Berichte.

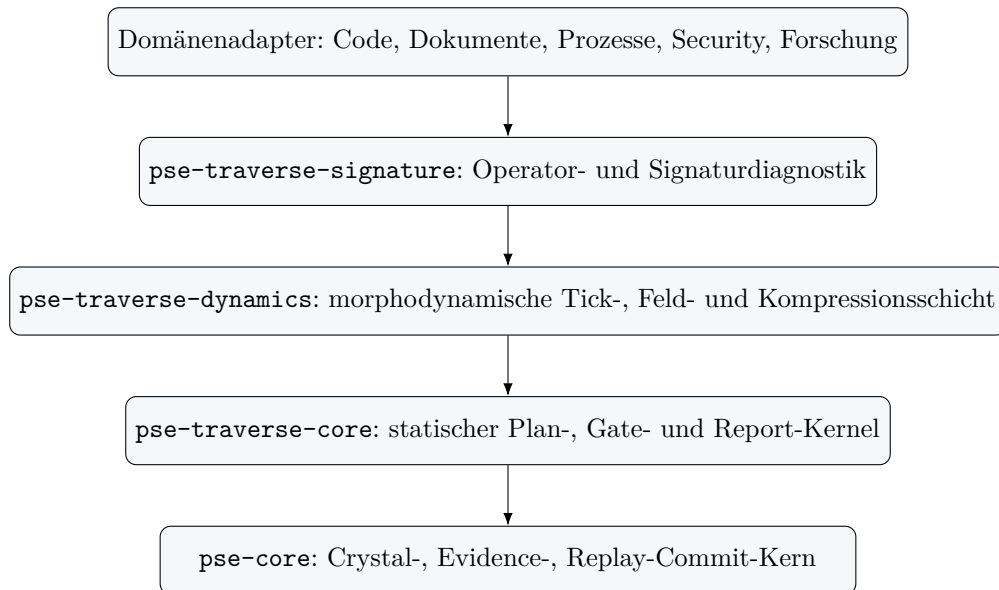
Folgende Begriffe sind im Core-Code MUST NOT auftauchen:

Dionice	Gabriel	APOLLYON
Metatron	Mandorla	MEF als Kernontologie
4D-Trichter	5D-Cube	QLogic als Pflichtbegriff
S7 als Pflichtstruktur	Engel-/Thron-Begriffe	metaphysische Gate-Namen

Zulässig sind Legacy-Hinweise in Dokumentation oder optionalen Adaptern, sofern sie nicht Teil des `pse-traverse-dynamics`-Kerns sind.

3 Systemrolle innerhalb der Gesamtarchitektur

3.1 Layer-Modell



`pse-traverse-dynamics` ist keine Konkurrenz zu `pse-traverse-signature`. Beide Schichten erfüllen unterschiedliche Aufgaben:

- `pse-traverse-signature` diagnostiziert die globale Struktur eines Graphen oder Operators.
- `pse-traverse-dynamics` stabilisiert und verdichtet den Zustand über getaktete Übergänge.
- `pse-traverse-core` erzeugt Pläne, Kandidaten, GateReports und RunReports.
- `pse-core` entscheidet über echte post-symbolische Crystal-Commits.

3.2 Integration in bestehende Pipeline

Vor der Dynamikschicht:

```
ProblemSpec -> FieldCube -> DoFGraph -> CollapsePlan -> Candidate -> GateReport ->
CommitOutcome
```

Mit optionaler Dynamikschicht:

```
ProblemSpec
-> FieldCube
-> DoFGraph
-> DynamicTraversalState
-> DynamicTick^n
-> StabilizedTraversalState
-> CollapsePlan / Candidate
-> GateReport + DynamicGateReport
-> CommitOutcome
```

DYN-PIPELINE-01

Die Dynamikschicht SHOULD vor der finalen Kandidatenerzeugung laufen, wenn ein Problemraum starke Kopplung, hohe Fragmentierung, hohe Candidate-Dichte oder instabile Planreihenfolgen aufweist. Sie MAY nachgelagert als Transition-Auditor verwendet werden, wenn bereits Kandidaten existieren.

4 Normative Sprache

Die Schlüsselwörter **MUST**, **MUST NOT**, **SHOULD** und **MAY** sind normativ zu verstehen:

- **MUST**: zwingende Implementierungsanforderung.
- **MUST NOT**: verbotene Implementierung.
- **SHOULD**: empfohlene Implementierung; Abweichung benötigt Begründung im Bericht.
- **MAY**: optionale Implementierung ohne Konformitätsverlust.

5 Kernabstraktionen

5.1 CanonicalNumber

Da `pse-traverse-dynamics` deterministisch und replayfähig sein muss, dürfen rohe `f64`-Werte nicht unkontrolliert in content-addressed Artefakte gelangen.

```
pub enum CanonicalNumber {
    FixedI64 { raw: i64, scale: u32 },
    Rational { num: i128, den: i128 },
}
```

NUMBER-01

Alle Werte, die in IDs, Hashes, Reports oder Replay-Prüfungen eingehen, **MUST** als `CanonicalNumber` serialisiert werden. Interne numerische Berechnung **MAY** temporär `f64` verwenden, sofern eine deterministische Quantisierung vor Persistenz erfolgt.

Empfohlene Standardquantisierung:

$$q(x) = \text{round}_{\text{banker}}(x \cdot 10^9) \quad (1)$$

und Speicherung als:

```
FixedI64 { raw: q(x), scale: 9 }
```

5.2 Hash256 und StableId

```
pub struct Hash256(pub [u8; 32]);
pub struct StableId(pub Hash256);
```

Alle IDs werden über kanonisch serialisierte Inhalte gebildet:

$$\text{id}(x) = \text{SHA256}(\text{JCS}(x)) \quad (2)$$

HASH-01

Kein Hash darf Wall-Clock-Zeit, nichtdeterministische UUIDs, HashMap-Iteration, Betriebssystempfade oder nichtkanonische Float-Serialisierung enthalten.

5.3 BaseState und LiftedState

```
pub struct BaseState {
    pub state_id: Hash256,
    pub coords: Vec<CanonicalNumber>,
    pub weight: CanonicalNumber,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub struct LiftedState {
    pub state_id: Hash256,
    pub base_state_id: Hash256,
    pub coords: Vec<CanonicalNumber>,
    pub auxiliary: CanonicalNumber,
    pub lift_profile: LiftProfile,
}
```

BaseState ist der domänenneutrale Prozesszustand. **LiftedState** ist derselbe Zustand in einem erweiterten Zustandsraum. Das Vorbild aus DioniceOS war $4D \rightarrow 5D$; im neutralen Kern gilt allgemein $N \rightarrow N + 1$.

```
pub struct LiftProfile {
    pub profile_id: Hash256,
    pub base_dimension: usize,
    pub lifted_dimension: usize,
    pub auxiliary_kind: AuxiliaryKind,
}

pub enum AuxiliaryKind {
    LogicalTick,
    Phase,
    SequenceIndex,
    PolicyEpoch,
}
```

5.4 StateLift

```
pub trait StateLift {
    fn lift(&self, base: &BaseState, tick: u64) -> LiftedState;
    fn project(&self, lifted: &LiftedState) -> BaseState;
    fn roundtrip_error(&self, base: &BaseState, tick: u64) -> CanonicalNumber;
}
```

LIFT-01

Jeder **StateLift** **MUST** einen Roundtrip-Test bereitstellen. Für einen Lift L und eine Projektion P muss gelten:

$$\epsilon_{rt}(s) = \|P(L(s, t)) - s\| \leq \epsilon_{max}$$

Der Standardwert für ϵ_{max} ist **1e-9** nach Quantisierung.

6 Field Layer

6.1 FieldSignal

Der Feldabsorber extrahiert aus einer Menge gelifteter Zustände ein abstraktes Signal:

```
pub struct FieldSignal {
    pub signal_id: Hash256,
    pub alignment: CanonicalNumber,
    pub dispersion: CanonicalNumber,
    pub pressure: CanonicalNumber,
    pub state_count: usize,
}
```

Semantik:

Feldwert	Bedeutung
<code>alignment</code>	Grad gemeinsamer Ausrichtung; neutralisierte Form des Resonanz-/Phasenfeldes.
<code>dispersion</code>	Streuung der Zustände; neutralisierte Form morphodynamischer Varianz.
<code>pressure</code>	kombinierter Kopplungsdruck; entscheidet, ob Split, Merge, Prune oder Gate-Verschärfung sinnvoll ist.
<code>state_count</code>	Anzahl der im Signal berücksichtigten Zustände.

6.2 FieldAbsorber

```
pub trait FieldAbsorber {
    fn absorb(&self, states: &[LiftedState], policy: &DynamicPolicy) -> FieldSignal;
}
```

Referenzalgorithmus:

1. Sortiere Zustände nach `state_id`.
2. Berechne `alignment` aus gewichteter Kohärenz der Hilfskoordinate.
3. Berechne `dispersion` als mittlere komponentenweise Varianz der Basisdimensionen.
4. Berechne `pressure = alpha * alignment + beta * dispersion`.
5. Quantisiere alle Werte kanonisch.
6. Bilde `signal_id` aus kanonischem Signalinhalt.

```
pub struct DefaultFieldAbsorber {
    pub alpha: CanonicalNumber,
    pub beta: CanonicalNumber,
}
```

FIELD-01

Ein leerer Zustandssatz MUST ein neutrales Signal erzeugen: `alignment = 0`, `dispersion = 0`, `pressure = 0`, `state_count = 0`. Das System darf nicht panicken.

7 Guidance Field

7.1 Datenmodell

```
pub struct GuidanceNode {
    pub id: StableId,
    pub state: LiftedState,
    pub resonance: CanonicalNumber,
    pub mass: CanonicalNumber,
}

pub struct GuidanceTransition {
    pub from: StableId,
    pub to: StableId,
    pub phase_delta: CanonicalNumber,
    pub coherence: CanonicalNumber,
    pub weight: CanonicalNumber,
}

pub struct GuidanceFieldState {
    pub nodes: BTreeMap<StableId, GuidanceNode>,
    pub transitions: Vec<GuidanceTransition>,
    pub field_id: Hash256,
}
```

7.2 GuidanceField-Trait

```
pub trait GuidanceField {
    fn relax(&mut self, signal: &FieldSignal, policy: &DynamicPolicy) ->
    GuidanceRelaxReport;
    fn gradient(&self, position: &LiftedState) -> LiftedState;
    fn snapshot(&self) -> GuidanceFieldState;
}
```

7.3 Relaxationsregeln

Die neutrale Relaxation folgt dem DioniceOS-Muster, aber ohne dessen Namensbindung:

1. Knotenausrichtung wird anhand des Feldsignals aktualisiert.
2. Transitionen erhalten **phase_delta** aus der Differenz ihrer Hilfskoordinaten.
3. Transition-Kohärenz sinkt bei hoher Differenz oder hoher Dispersion.
4. Transitionen unter **prune_threshold** werden entfernt.
5. Alle Vektoren werden nach **from**, **to**, **weight** sortiert.

```
pub struct GuidanceRelaxReport {
    pub report_id: Hash256,
    pub field_before: Hash256,
    pub field_after: Hash256,
```

```

    pub nodes_updated: usize,
    pub transitions_updated: usize,
    pub transitions_pruned: usize,
}

```

GUIDE-01

`gradient(position)` MUST für leere Felder einen Nullgradienten mit derselben Dimension wie `position` zurückgeben. Keine Funktion im Guidance Layer darf aufgrund leerer Zustandsmengen abbrechen.

8 Morphodynamic Compressor

8.1 Kompressionsmodell

Der Kompressor ist ein dynamischer Graphspeicher. Er sammelt Zustände, verstärkt kohärente Übergänge, splittet divergente Bereiche, merged konvergente Bereiche und pruned schwache Kanten.

```

pub struct CompressionNode {
    pub id: StableId,
    pub state: LiftedState,
    pub mass: CanonicalNumber,
    pub variance: CanonicalNumber,
    pub born_tick: u64,
}

pub struct CompressionEdge {
    pub from: StableId,
    pub to: StableId,
    pub weight: CanonicalNumber,
    pub decay: CanonicalNumber,
    pub coherence: CanonicalNumber,
}

pub struct CompressionGraph {
    pub nodes: BTreeMap<StableId, CompressionNode>,
    pub edges: Vec<CompressionEdge>,
    pub graph_id: Hash256,
}

```

8.2 Kompressor-Trait

```

pub trait MorphodynamicCompressor {
    fn add_state(&mut self, state: LiftedState) -> StableId;
    fn advect(&mut self, guidance: &BaseState, policy: &DynamicPolicy, tick: u64) -> CompressorStats;
    fn snapshot(&self) -> CompressionGraph;
}

```

8.3 CompressorStats

```
pub struct CompressorStats {
    pub stats_id: Hash256,
    pub nodes_created: usize,
    pub nodes_merged: usize,
    pub nodes_split: usize,
    pub edges_created: usize,
    pub edges_pruned: usize,
    pub density: CanonicalNumber,
    pub graph_before: Hash256,
    pub graph_after: Hash256,
}
```

8.4 Hebbian Update

Für jede Kante e_{ij} :

$$w_{ij}^{t+1} = \max \left(0, w_{ij}^t + \alpha \cdot c_{ij} \cdot u_{ij} - d \right) \quad (3)$$

mit:

- α : Policy-Lernrate,
- c_{ij} : Kohärenz zwischen Zuständen,
- u_{ij} : gemeinsame Nutzung / Massenkopplung,
- d : Decay.

COMPRESS-01

Split, Merge und Prune MUST deterministisch sein. Wenn mehrere Operationen gleichzeitig möglich sind, entscheidet die lexikographische Ordnung der content-addressed IDs. Implementierungen MUST NOT über ungeordnete Maps iterieren, wenn daraus persistente Reihenfolgen entstehen.

8.5 Split

Ein Knoten n wird gesplittet, wenn:

$$\text{mass}(n) > \theta_{\text{split}} \wedge \text{variance}(n) > \theta_{\text{variance}} \quad (4)$$

Der Split erzeugt zwei oder mehr deterministische Nachfolger. Die Standardimplementierung erzeugt zwei Nachfolger entlang der stärksten Varianzachse.

8.6 Merge

Zwei Knoten i, j werden gemerged, wenn:

$$\text{dist}(i, j) \leq \theta_{\text{merge_dist}} \wedge \text{coherence}(i, j) \geq \theta_{\text{merge_coherence}} \quad (5)$$

Der neue Zustand ist ein massengewichteter Mittelzustand, quantisiert nach `CanonicalNumber`.

8.7 Prune

Eine Kante wird entfernt, wenn:

$$w_{ij} < \theta_{prune} \quad (6)$$

Pruning ist kein Fehler. Es ist eine normale Kompressionsoperation und muss im `CompressorStats`-Bericht sichtbar sein.

9 Dynamic Policy

9.1 Policy-Klassen

```
pub enum DynamicPolicyKind {
    Explore,
    Exploit,
    Homeostasis,
}

pub struct DynamicPolicy {
    pub policy_id: Hash256,
    pub kind: DynamicPolicyKind,
    pub learning_rate: CanonicalNumber,
    pub decay: CanonicalNumber,
    pub split_threshold: CanonicalNumber,
    pub merge_threshold: CanonicalNumber,
    pub prune_threshold: CanonicalNumber,
    pub variance_threshold: CanonicalNumber,
    pub strict_coherence: bool,
    pub target_density: Option<CanonicalNumber>,
}
```

9.2 Semantik

Policy	Semantik
Explore	Erhält Vielfalt. Niedrige Prune- und Merge-Strenge. Geeignet für frühe oder fragmentierte Suchräume.
Exploit	Konsolidiert bekannte stabile Bereiche. Höhere Merge-/Prune-Strenge. Geeignet nach erfolgreicher Diagnose oder stabiler Gate-Historie.
Homeostasis	Hält Dichte und Komplexität in Zielgrenzen. Geeignet gegen Suchraumexplosion oder Kollaps auf zu wenige Alternativen.

9.3 Homeostasis-Adaption

Wenn `target_density` gesetzt ist:

$$rho_{ratio} = \frac{\rho_{current}}{\rho_{target}} \quad (7)$$

Regeln:

- Wenn $\rho_{ratio} > 1.2$: Decay und Prune-Schwelle erhöhen, Lernrate senken.
- Wenn $\rho_{ratio} < 0.8$: Decay und Prune-Schwelle senken, Lernrate erhöhen.
- Wenn $0.8 \leq \rho_{ratio} \leq 1.2$: Parameter unverändert lassen.

POLICY-01

Policy-Adaption MUST selbst deterministisch und replaybar sein. Sie darf nicht von Wall-Clock-Zeit, Thread-Scheduling oder zufälliger Reihenfolge abhängen.

10 Transition Proof und Dynamic Gate

10.1 TransitionProof

```
pub struct TransitionProof {
    pub proof_id: Hash256,
    pub tick: u64,
    pub prev_state_root: Hash256,
    pub next_state_root: Hash256,
    pub path_delta: CanonicalNumber,
    pub alignment: CanonicalNumber,
    pub energy_delta: CanonicalNumber,
    pub density_delta: CanonicalNumber,
    pub valid: bool,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

10.2 Berechnung

Für Zustandsmengen S_t und S_{t+1} :

$$\Delta_{path} = \text{mean}_i (\|s_{i,t+1} - s_{i,t}\|) \quad (8)$$

$$\Delta_E = E(S_{t+1}) - E(S_t) \quad (9)$$

mit Standardenergie:

$$E(S) = \text{mean}_{s \in S} (\|s\|^2) \quad (10)$$

Die Alignment-Komponente wird aus `FieldSignal.alignment` übernommen.

10.3 DynamicGate

```
pub enum DynamicGateDecision {
    Fire,
    Hold,
}
```

```
pub struct DynamicGateConfig {
    pub config_id: Hash256,
    pub max_path_delta: CanonicalNumber,
    pub min_alignment: CanonicalNumber,
    pub max_density_delta: Option<CanonicalNumber>,
    pub require_energy_decrease: bool,
}

pub struct DynamicGateReport {
    pub report_id: Hash256,
    pub proof_id: Hash256,
    pub decision: DynamicGateDecision,
    pub passed: bool,
    pub reasons: Vec<GateReason>,
}
```

Standardbedingung:

$$\text{Fire} \iff \text{valid} \wedge \Delta_{\text{path}} \leq \epsilon \wedge \text{alignment} \geq \phi_{\min} \wedge (\Delta_E < 0 \vee \text{require_energy_decrease}) \quad (11)$$

GATE-01

Ein fehlgeschlagenes DynamicGate MUST fail-closed sein. Es darf keinen Commit und keinen finalen Candidate erzwingen. Es erzeugt einen DynamicGateReport mit Hold und expliziten Gründen.

11 Dynamic Tick

11.1 State Container

```
pub struct DynamicTraversalState {
    pub state_id: Hash256,
    pub tick: u64,
    pub base_states: Vec<BaseState>,
    pub guidance_field: GuidanceFieldState,
    pub compression_graph: CompressionGraph,
    pub policy: DynamicPolicy,
    pub descriptor: DynamicRunDescriptor,
}
```

11.2 Input und Report

```
pub struct DynamicTickInput {
    pub tick: u64,
    pub base_states: Vec<BaseState>,
    pub policy: DynamicPolicy,
    pub gate_config: DynamicGateConfig,
    pub compute_proof: bool,
}
```

```
pub struct DynamicTickReport {
    pub report_id: Hash256,
    pub tick: u64,
    pub prev_state_id: Hash256,
    pub next_state_id: Hash256,
    pub field_signal: FieldSignal,
    pub guidance_relax: GuidanceRelaxReport,
    pub compressor_stats: CompressorStats,
    pub transition_proof: Option<TransitionProof>,
    pub gate_report: DynamicGateReport,
    pub next_states: Vec<BaseState>,
}
```

11.3 Algorithmus

```
Algorithm DYNAMIC_TICK(input, dyn_state):
    require input.tick == dyn_state.tick + 1 OR dyn_state.tick == 0

    1. Sort input.base_states by state_id.
    2. lifted_states := map lift(base_state, input.tick).
    3. field_signal := FieldAbsorber.absorb(lifted_states, policy).
    4. guidance_relax := GuidanceField.relax(field_signal, policy).
    5. gradients := map GuidanceField.gradient(lifted_state).
    6. guidance := project aggregate(gradients) into BaseState.
    7. Add lifted_states to MorphodynamicCompressor.
    8. compressor_stats := compressor.advect(guidance, policy, tick).
    9. next_states := project selected compressor nodes to BaseState.
    10. If compute_proof:
        proof := TransitionProof(prev_states, next_states, field_signal, stats)
    Else:
        proof := None with evidence reason.
    11. gate_report := DynamicGate.evaluate(proof, gate_config).
    12. dyn_state := update with next_states, field, compressor, policy.
    13. Return DynamicTickReport with content-addressed report_id.
```

TICK-01

Der Tick muss total sein: Für leere Eingaben, leere Felder oder fehlende Transitionen muss ein gültiger Bericht entstehen. Der Bericht darf Hold enthalten, aber die Pipeline darf nicht abbrechen, außer bei Schema-, Canonicalization- oder Speicherfehlern.

12 Interaction mit Signature Layer

pse-traverse-signature und pse-traverse-dynamics sollen zusammenarbeiten, aber lose gekoppelt bleiben.

```
pub struct SignatureDrivenPolicyHint {
    pub source_diagnostics_id: Hash256,
    pub recommended_policy: DynamicPolicyKind,
    pub confidence: CanonicalNumber,
    pub reason: String,
```

}

Mapping-Beispiele:

Signature-Befund	Policy-Hint	Begründung
Fragmentiert / uncoupled	Explore	Suchraum benötigt Diversität und weitere Erkundung.
SymmetricCoupled	Homeostasis	Kopplung ist vorhanden, aber kann dicht werden; Dichte stabilisieren.
DirectedCoupled	Exploit	Reihenfolge und Kausalität sind wichtig; strenge Kohärenz.
Degenerate	Explore + Split	Zu viele äquivalente Freiheitsgrade; Divergenz erzeugen.
Stable / high coherence	Exploit	Kandidaten konsolidieren und Gate verschärfen.

SIGNATURE-INTEGRATION-01

Policy-Hints aus `pse-traverse-signature` SHOULD erklärbar und im `DynamicRunDescriptor` referenziert sein. Sie dürfen die Policy nicht stillschweigend ändern. Jede automatische Policy-Änderung MUST im Report erscheinen.

13 Integration mit `pse-traverse-core`

13.1 Erweiterte Run-Struktur

```
pub struct TraversalRunReport {
    // existing fields
    pub problem_spec: ProblemSpec,
    pub field_cube: FieldCube,
    pub dof_graph: DoFGraph,
    pub collapse_plan: CollapsePlan,
    pub gate_report: GateReport,
    pub commit_outcome: CommitOutcome,

    // new optional dynamic fields
    pub dynamic_run: Option<DynamicRunReport>,
}

pub struct DynamicRunReport {
    pub run_id: Hash256,
    pub descriptor: DynamicRunDescriptor,
    pub ticks: Vec<DynamicTickReport>,
    pub final_state_id: Hash256,
    pub final_gate_decision: DynamicGateDecision,
    pub replay_address: Hash256,
}
```

13.2 CollapsePlan-Einfluss

pse-traverse-dynamics darf CollapsePlan beeinflussen über:

- Priorisierung stabiler Knotencluster,
- Excision-Hinweise bei instabilen Übergängen,
- Merge-Vorschläge für konvergente Freiheitsgrade,
- Split-Vorschläge für hochvariante Freiheitsgrade,
- Gate-Schärfung bei hoher Dispersion oder hoher Dichte.

Kein impliziter Plan-Override

pse-traverse-dynamics darf einen existierenden CollapsePlan nicht stillschweigend überschreiben. Jede Planänderung muss als PlanDelta serialisiert, begründet und content-addressed werden.

```
pub struct PlanDelta {
    pub delta_id: Hash256,
    pub source_dynamic_report: Hash256,
    pub before_plan_id: Hash256,
    pub after_plan_id: Hash256,
    pub operations: Vec<PlanDeltaOp>,
    pub reasons: Vec<String>,
}

pub enum PlanDeltaOp {
    ReorderStep { step_id: Hash256, before_index: usize, after_index: usize },
    SplitStep { step_id: Hash256, new_steps: Vec<Hash256> },
    MergeSteps { step_ids: Vec<Hash256>, merged_step: Hash256 },
    TightenGate { gate_key: String, old_value: CanonicalNumber, new_value: CanonicalNumber },
    AddExcisionHint { target_id: Hash256, reason: String },
}
```

14 Replay und Determinismus

14.1 DynamicRunDescriptor

```
pub struct DynamicRunDescriptor {
    pub descriptor_id: Hash256,
    pub schema_version: String,
    pub operator_versions: BTreeMap<String, String>,
    pub initial_state_root: Hash256,
    pub policy_schedule: Vec<PolicyScheduleEntry>,
    pub gate_config: DynamicGateConfig,
    pub quantization: QuantizationPolicy,
    pub max_ticks: u64,
    pub stop_conditions: Vec<DynamicStopCondition>,
}
```

14.2 Stop Conditions

```
pub enum DynamicStopCondition {
    MaxTicks(u64),
    GateFireCount { min_count: u64 },
    StabilizedDensity { tolerance: CanonicalNumber, consecutive_ticks: u64 },
    StabilizedStateRoot { consecutive_ticks: u64 },
    MaxHoldCount { max_count: u64 },
}
```

14.3 Replay-Regel

REPLAY-01

Gleicher `DynamicRunDescriptor` plus gleiche Anfangszustände plus gleiche Operatorversionen **MUST** byte-identische `DynamicRunReports` erzeugen. Wenn dies nicht gilt, ist die Implementierung nicht konform.

14.4 Nichtdeterminismus-Ausschlüsse

Verboten in content-addressed Ausgaben:

- `HashMap`-Iteration ohne Sortierung,
- Wall-Clock-Zeit,
- Thread-Reihenfolge,
- nichtgesetzte Random Seeds,
- Plattform-abhängige Float-Formatierung,
- Dateisystem-Reihenfolge,
- nichtkanonische JSON-Ausgabe.

15 Fehlersemantik

```
pub enum DynamicError {
    CanonicalizationFailed { reason: String },
    InvalidDimension { expected: usize, got: usize },
    NonFiniteNumber { field: String },
    ReplayMismatch { expected: Hash256, actual: Hash256 },
    OperatorVersionMismatch { expected: String, actual: String },
    StorageFailure { reason: String },
}
```

ERROR-01

Normale Gate-Holds, leere Zustandsmengen, Pruning und No-Stabilization sind keine technischen Fehler. Sie sind valide Diagnoseergebnisse. Technische Fehler betreffen nur Schema, Canonicalization, nichtfinite Zahlen, Versionsmismatch, Replay-Mismatch oder Speicherfehler.

16 CLI-Spezifikation

pse-traverse-dynamics soll über die bestehende pse-traverse-cli erreichbar sein.

16.1 Subcommands

```
pse-traverse-cli dynamics init    --problem problem.json --out dyn_init.json
pse-traverse-cli dynamics tick    --state dyn_state.json --out tick.json
pse-traverse-cli dynamics run     --problem problem.json --ticks 32 --out dyn_run.json
pse-traverse-cli dynamics replay  --report dyn_run.json
pse-traverse-cli dynamics inspect --report dyn_run.json
```

16.2 CLI-Anforderungen

- `init` erzeugt einen initialen `DynamicTraversalState`.
- `tick` führt exakt einen Tick aus.
- `run` führt Ticks bis `max_ticks` oder Stop Condition aus.
- `replay` rekonstruiert alle `TickReports` und vergleicht Byte-Identität.
- `inspect` gibt menschenlesbare, nichtkanonische Diagnose aus; diese Ausgabe darf nicht für Hashes verwendet werden.

17 Testplan

17.1 Unit Tests

Test	Erwartung
<code>lift_roundtrip_preserves_base</code>	Projektion des gelifteten Zustands entspricht dem Ausgangszustand innerhalb Quantisierung.
<code>field_empty_is_neutral</code>	Leere Eingabe erzeugt neutrales <code>FieldSignal</code> .
<code>field_absorb_is_deterministic</code>	Gleiche Zustände in anderer Reihenfolge ergeben identisches Signal.
<code>guidance_empty_gradient_zero</code>	Leeres Feld erzeugt Nullgradient.
<code>compressor_split_deterministic</code>	Split-Ausgabe ist bei gleichen Inputs byte-identisch.
<code>compressor_merge_deterministic</code>	Merge-Reihenfolge hängt nur von <code>StableIds</code> ab.
<code>policy_homeostasis_adapts_high_density</code>	Hochere Dichte erhöht Decay/Prune und senkt Lernrate.
<code>policy_homeostasis_adapts_low_density</code>	Niedrigere Dichte senkt Decay/Prune und erhöht Lernrate.
<code>transition_proof_detects_path_delta</code>	<code>Delta</code> steigt bei großer Zustandsänderung.
<code>dynamic_gate_hold_on_invalid_proof</code>	Ungültiger Proof erzeugt Hold.
<code>dynamic_tick_empty_input_total</code>	Leere Eingabe erzeugt gültigen <code>TickReport</code> .

17.2 Integration Tests

Test	Erwartung
<code>cli_dynamics_run_writes_report</code>	CLI erzeugt vollständigen <code>DynamicRunReport</code> .
<code>cli_dynamics_replay_ok</code>	Replay akzeptiert eigenen Report.
<code>cli_dynamics_byte_identical</code>	Zwei Runs mit gleichem Descriptor sind byte-identisch.
<code>dynamic_plan_delta_is_explicit</code>	Jede Planänderung erzeugt <code>PlanDelta</code> .
<code>dynamic_gate_integrates_gate_report</code>	<code>DynamicGateReport</code> erscheint im <code>TraversalRunReport</code> .
<code>pse_bridge_still_exclusive</code>	Kein Dynamic-Modul erzeugt <code>SemanticCrystal</code> direkt.

17.3 Property Tests

- Für alle endlichen Zustandsvektoren gilt: Canonicalization erzeugt keine nichtfiniten Zahlen.
- Für alle Permutationen der Eingabezustände gilt: sortierte Verarbeitung erzeugt identische IDs.
- Für alle leeren oder Ein-Zustand-Eingaben gilt: Tick ist total und panict nicht.
- Für alle Reports gilt: `report_id` ändert sich, wenn sich ein normativer Inhalt ändert.

18 Definition of Done

Eine Implementierung von `pse-traverse-dynamics` ist v0.1-konform, wenn alle folgenden Punkte erfüllt sind:

DoD v0.1

1. `cargo test -workspace` besteht.
2. Alle öffentlichen Typen sind dokumentiert.
3. Kein Core-Typ enthält Dionice-, Gabriel-, APOLLYON-, Metatron-, Mandorla- oder MEF-Pflichtbegriffe.
4. `BaseState`, `LiftedState`, `FieldSignal`, `GuidanceFieldState`, `CompressionGraph`, `TransitionProof`, `DynamicGateReport`, `DynamicTickReport` sind content-addressed.
5. `dynamic tick` ist total für leere Eingaben.
6. `dynamic run` ist byte-identisch replaybar.
7. Homeostasis-Policy verändert Parameter deterministisch anhand von Dichte.
8. `DynamicGate` ist fail-closed.
9. Jede Planänderung wird als `PlanDelta` dokumentiert.
10. Dynamic Layer erzeugt keine `SemanticCrystals`.
11. PSE-Bridge bleibt einzige Commit-Quelle für echte Crystals.
12. CLI unterstützt `dynamics init|tick|run|replay|inspect`.
13. Mindestens 10 Unit Tests, 4 Integration Tests und 3 Property Tests decken die Kernmechanik ab.

19 Empfohlene Dateistruktur

```
crates/pse-traverse/src/
dynamic_state.rs      // BaseState, LiftedState, StateLift
field.rs              // FieldSignal, FieldAbsorber
guidance_field.rs     // GuidanceNode, GuidanceTransition, GuidanceField
compressor.rs         // CompressionGraph, MorphodynamicCompressor
dynamic_policy.rs     // Explore, Exploit, Homeostasis
transition_proof.rs   // TransitionProof, DynamicGate
dynamic_tick.rs       // DynamicTraversalState, DynamicTickReport
dynamic_report.rs     // DynamicRunReport, replay identity
plan_delta.rs         // Explicit dynamic influence on CollapsePlan
```

20 Coding-Agent-Auftrag

Der folgende Auftrag ist als direkte Schablone für einen Coding Agent geeignet.

PATCH-ID: PSE-TRAVERSE-DYNAMICS-01

OBJECTIVE:

Implement an optional, deterministic, domain-agnostic morphodynamic traversal layer in `crates/pse-traverse`. Extract only the neutral mechanics inspired by

DioniceOS: state lift/projection, field absorption, guidance relaxation, morphodynamic compression, dynamic policy, transition proof, dynamic gate, replayable tick reports, and CLI support.

MUST:

- Add modules:
 - dynamic_state.rs
 - field.rs
 - guidance_field.rs
 - compressor.rs
 - dynamic_policy.rs
 - transition_proof.rs
 - dynamic_tick.rs
 - dynamic_report.rs
 - plan_delta.rs
- Use BTreeMap or sorted Vecs for all canonical output.
- Quantize all report-relevant numeric values before hashing.
- Make DynamicTick total for empty inputs.
- Make DynamicGate fail-closed.
- Content-address all dynamic artefacts with SHA-256 over canonical bytes.
- Embed DynamicRunReport optionally into TraversalRunReport.
- Add CLI subcommands: dynamics init, tick, run, replay, inspect.
- Add tests proving byte-identical replay.
- Ensure no module fabricates SemanticCrystal.

MUST NOT:

- Introduce Dionice, Gabriel, APOLLYON, Metatron, Mandorla, MEF, 4D-Trichter, or 5D-Cube as core concepts.
- Depend on wall-clock time for hashes or reports.
- Use HashMap iteration for persisted output order.
- Use non-seeded randomness.
- Replace the PSE bridge.
- Generate SemanticCrystal outside the existing PSE macro_step bridge.

SHOULD:

- Reuse existing canonicalization utilities from pse-traverse.
- Integrate DynamicGateReport into GateReport as an optional evidence channel.
- Allow SignatureDiagnostics to recommend DynamicPolicy through explicit hints.
- Keep the dynamic layer feature-gated if it increases compile cost.

DONE WHEN:

- cargo test --workspace passes.
- CLI dynamic run produces a report.
- CLI replay verifies byte identity.
- Empty-input tick returns a valid Hold report.
- Gate failure never commits.
- Dynamic layer remains domain-agnostic and terminology-clean.

21 Abschließende Architekturformel

Die extrahierte DioniceOS-Topologie reduziert sich auf folgende neutrale Formel:

$$\text{DynamicTick}(S_t) = \text{Gate}(\text{Proof}(\text{Compress}(\text{Project}(\text{Gradient}(\text{Relax}(\text{Absorb}(\text{Lift}(S_t))))))) \quad (12)$$

In Worten:

Ein Zustandsraum wird erweitert, feldförmig vermessen, über einen Gradienten geführt, morphodynamisch komprimiert, durch einen Transition-Proof geprüft und fail-closed gegated. Nur die resultierenden Berichte, Zustände und Gate-Signale fließen in `pse-traverse-core` ein; echte Crystals entstehen ausschließlich über `pse-core`.

Das ist der reine, domänenagnostische Kristall aus DioniceOS, geeignet als Implementierungsgrundstein für den nächsten Ausbau von `pse-traverse`.

A Glossar

Begriff	Definition
<code>BaseState</code>	Prozesszustand im domänenneutralen Basisraum.
<code>LiftedState</code>	Erweiterter Zustand mit zusätzlicher Hilfskoordinate.
<code>FieldSignal</code>	Aggregiertes Feldsignal aus Alignment, Dispersion und Pressure.
<code>GuidanceField</code>	Dynamisches Feld, das Gradienten zur Zustandsführung erzeugt.
<code>MorphodynamicCompressor</code>	Graphbasierter Kompressor mit Add, Advect, Split, Merge und Prune.
<code>DynamicPolicy</code>	Steuerprofil für Exploration, Exploitation oder Homeostasis.
<code>TransitionProof</code>	Prüfnachweis über Stabilität und Gültigkeit eines Zustandsübergangs.
<code>DynamicGateReport</code>	Fail-closed Gate-Bericht über Fire/Hold-Entscheidung.
<code>DynamicTickReport</code>	Content-addressed Bericht eines einzelnen dynamischen Ticks.
<code>PlanDelta</code>	Explizite, begründete Änderung eines CollapsePlans durch dynamische Evidenz.
<code>SemanticCrystal</code>	PSE-Artefakt; darf nicht von <code>pse-traverse-dynamics</code> erzeugt werden.

B Konformitätsmatrix

ID	Anforderung	Pflichtgrad
NEUTRAL-CORE-01	Core ohne DioniceOS-spezifische Eigennamen.	MUST
NUMBER-01	Report-relevante Zahlen kanonisch serialisieren.	MUST
HASH-01	Keine nichtdeterministischen Werte in Hashes.	MUST

LIFT-01	Jeder Lift hat Roundtrip-Test.	MUST
FIELD-01	Leere Zustände erzeugen neutrales Signal.	MUST
GUIDE-01	Leeres GuidanceField erzeugt Nullgradient.	MUST
COMPRESS-01	Split/Merge/Prune deterministisch.	MUST
POLICY-01	Policy-Adaption replayfähig.	MUST
GATE-01	DynamicGate fail-closed.	MUST
TICK-01	DynamicTick total.	MUST
REPLAY-01	Gleiche Inputs erzeugen byte-identische Reports.	MUST
ERROR-01	Gate-Hold ist Diagnose, kein technischer Fehler.	MUST
